Niezawodność systemów informatycznych

Zajęcia projektowe nr 1: Blok P1
Pomiar niezawodności testowego oprogramowania

Semestr 7, rok akademicki 2026/2027

1 Informacje organizacyjne

- **Blok tematyczny z karty zajęć:** P1: Pomiar niezawodności testowego oprogramowania
- **Forma pracy:** zespoły 2–3-osobowe, każdy zespół pracuje przez cały semestr na własnym systemie testowanym (SUT: *System Under Test*)

Cel dzisiejszych zajęć

Po dzisiejszych zajęciach każdy zespół powinien:

1. rozumieć i umieć zastosować podstawowe pojęcia niezawodności (fault, error, failure, MTTF, MTBF, MTTR, dostępność),
2. mieć uruchomiony własny system testowany (SUT) wraz z mechanizmem logowania awarii,
3. zebrać pierwszy zestaw danych i policzyć wstępne wskaźniki niezawodności.

Jak te zajęcia wpisują się w cały semestr

To pierwszy krok w projekcie, który będzie trwał przez cały semestr na tym samym SUT. Kolejne etapy to: testy strukturalne (pokrycie kodu, testy mutacyjne), testy funkcjonalne i niefunkcjonalne, analiza statyczna, a na koniec automatyzacja testów i szacowanie niezawodności za pomocą modeli wzrostu niezawodności. Dzisiejszy pomiar będzie waszym punktem odniesienia (*baseline*) do porównań na każdym kolejnym etapie.

2 Wprowadzenie teoretyczne

2.1 Fault, error, failure: trzy różne rzeczy

Fault, error, failure

Fault (usterka): błąd w kodzie lub projekcie, np. literówka w warunku logicznym. Usterka może istnieć w kodzie latami, jeśli nigdy nie zostanie wykonana.
Error (błąd stanu): nieprawidłowy stan wewnętrzny programu powstały w wyniku aktywacji usterki (np. zmienna przyjmuje błędną wartość).
Failure (awaria): obserwowalne odstępstwo zachowania systemu od specyfikacji, widoczne dla użytkownika.

Wskazówka

Łańcuch przyczynowy wygląda tak: **fault** → (aktywacja) → **error** → (propagacja) → **failure**. Usterka nie zawsze prowadzi do awarii: jeśli błędny fragment kodu nigdy nie zostanie wykonany albo błędny stan zostanie "zamaskowany", awaria nie wystąpi. To dlatego program z usterkami może działać poprawnie przez długi czas.

Częsty błąd studentów

Traktowanie liczby usterek (faults) w kodzie jako bezpośredniej miary niezawodności. Dwa programy mogą mieć tyle samo usterek, a bardzo różną niezawodność: zależy to od tego, jak często usterki są aktywowane w typowym użytkowaniu (patrz: profil operacyjny, sekcja 2.3).

2.2 Podstawowe miary niezawodności

MTTF: Mean Time To Failure

Średni czas do wystąpienia awarii, liczony dla systemów nienaprawialnych lub do pierwszej awarii:

$$MTTF = \frac{1}{n}\sum_{i=1}^{n} t_i$$

gdzie t_i to czas działania do i-tej obserwowanej awarii.

MTBF: Mean Time Between Failures

Średni czas pomiędzy kolejnymi awariami w systemie naprawialnym:

$$MTBF = MTTF + MTTR$$

MTTR: Mean Time To Repair

Średni czas potrzebny na usunięcie awarii (naprawę) i przywrócenie systemu do działania.

Dostępność: Availability

Prawdopodobieństwo, że system działa poprawnie w losowo wybranej chwili:

$$A = \frac{MTBF}{MTBF + MTTR} = \frac{MTTF}{MTTF + MTTR}$$

Dostępność często wyrażana jest w "dziewiątkach": np. 99,9% dostępności to ok. 8,76 godziny przestoju rocznie.

Skąd biorą się te 8,76 godziny

Rok ma $365 \times 24 = 8760$ godzin. Przy dostępności 99,9% systemowi "wolno" nie działać przez pozostałą 0,1% tego czasu, czyli jedną tysięczną:

$$0{,}001 \times 8760 \text{ h} = 8{,}76 \text{ h}$$

Ten sam mechanizm działa dla każdej liczby "dziewiątek": im więcej dziewiątek po przecinku, tym mniejszy ułamek roku zostaje na przestój:

Dostępność	Ułamek przestoju	Dopuszczalny przestój / rok
99%	0,01	ok. 3,65 dnia
99,9%	0,001	ok. 8,76 godziny
99,99%	0,0001	ok. 52,6 minuty
99,999%	0,00001	ok. 5,26 minuty

Każda kolejna ”dziewiątka” to przestój ok. 10 razy krótszy. Jednocześnie każda kolejna dziewiątka w praktyce kosztuje nieproporcjonalnie więcej (redundancja, monitoring, automatyczne failovery), więc wybór docelowego poziomu dostępności to decyzja biznesowa, nie tylko techniczna.

Uwaga

MTBF $\neq$ gwarancja, że urządzenie przeżyje tyle czasu. To średnia statystyczna z populacji obserwacji, nie prognoza dla pojedynczego egzemplarza. MTBF rzędu miliona godzin nie oznacza, że pojedynczy dysk przetrwa 114 lat: oznacza, że w dużej populacji dysków średni czas między awariami wynosi tyle godzin.

2.3 Profil operacyjny

Profil operacyjny

Opis tego, jak faktycznie używany jest system: które funkcje są wywoływane, jak często i w jakich proporcjach. Profil operacyjny pozwala testować system tak, jak będzie używany w praktyce, zamiast testować wszystkie funkcje ”po równo”.

Wskazówka

Jeśli 80% ruchu w systemie e-commerce to przeglądanie katalogu, a 5% to finalizacja zamówienia, to testowanie powinno odzwierciedlać te proporcje: awaria w rzadko używanej funkcji ma mniejszy wpływ na odczuwaną niezawodność niż awaria w ścieżce używanej przez większość użytkowników, nawet jeśli obie funkcje mają po jednej usterce.

2.4 Klasyfikacja dotkliwości awarii

Severity: dotkliwość awarii

Nie każda awaria jest tak samo ważna. Przy logowaniu i raportowaniu klasyfikujemy awarie wg wpływu na system i użytkownika:

Poziom	Opis	Przykład
Krytyczna	System przestaje działać, dane są tracone lub uszkodzone, awaria bezpieczeństwa	Przelew znika bez śladu; saldo konta ujemne bez limitu
Poważna	Funkcja nie działa poprawnie, ale system jako całość pozostaje stabilny	Błędne zaokrąglenie odsetek; brak komunikatu o błędzie przy niepoprawnych danych
Drobna	Odstępstwo kosmetyczne lub w rzadkiej ścieżce, nie wpływa na wynik funkcjonalny	Literówka w komunikacie; złe formatowanie daty w historii transakcji

Wskazówka

Klasyfikacja dotkliwości jest subiektywna, ale musi być konsekwentna w obrębie jednego zespołu: przyjmijcie wspólną definicję na początku projektu i trzymajcie się jej przez cały semestr, żeby wasze dane dawały się porównywać między kolejnymi etapami.

2.5 Intensywność awarii i gęstość defektów

Failure intensity, defect density

Intensywność awarii $\lambda(t)$: liczba awarii na jednostkę czasu (lub na jednostkę wykonań). W miarę usuwania usterek zwykle maleje: to podstawa modeli wzrostu niezawodności, którymi zajmiemy się w bloku P5.
Gęstość defektów: liczba znalezionych usterek na tysiąc linii kodu (defects / KLOC). Używana do porównywania jakości modułów względem siebie, nie jako wartość absolutna.

Therac-25

Aparat do radioterapii Therac-25 (lata 80.) spowodował śmiertelne przedawkowania promieniowania. Przyczyną był tzw. *race condition*: błąd ujawniający się tylko wtedy, gdy operator wprowadzał dane wystarczająco szybko, by ”wyścignąć” aktualizację stanu programu. Usterka istniała od dawna, ale aktywowała się tylko przy konkretnym, rzadkim profilu użytkowania.

Ariane 5

W 1996 roku rakieta Ariane 5 uległa samozniszczeniu 37 sekund po starcie. Przyczyną było przepełnienie przy konwersji 64-bitowej liczby zmiennoprzecinkowej na 16-bitową liczbę całkowitą: w module odziedziczonym z Ariane 4, gdzie warunki lotu nigdy nie prowadziły do takiej wartości. Ten sam kod, inny profil operacyjny: inny skutek.

Knight Capital

W 2012 roku firma Knight Capital straciła ok. 440 mln dolarów w 45 minut. Przyczyną było ponowne użycie starej flagi konfiguracyjnej na jednym z ośmiu serwerów produkcyjnych:

uśpiony, nieaktywowany od lat fragment kodu nagle zaczął wykonywać testowe transakcje na prawdziwym rynku.

3 Metodyka: jak mierzyć niezawodność w projekcie

W projekcie niezawodność będziemy mierzyć empirycznie: na podstawie obserwowanych awarii podczas testowania waszego SUT.

Co zapisujemy przy każdej awarii

- znacznik czasu (timestamp),
- wywoływana funkcja / moduł / endpoint,
- rodzaj awarii (wyjątek, złe dane wyjściowe, zawieszenie, naruszenie bezpieczeństwa itd.),
- dotkliwość (severity: krytyczna / poważna / drobna),
- dane wejściowe, które spowodowały awarię (do reprodukcji).

Prosty logger awarii: przykład w Pythonie

```python
import time
import json
import functools

def niezawodnosc_logger(log_path="awarie.jsonl"):
    def dekorator(funkcja):
        @functools.wraps(funkcja)
        def wrapper(*args, **kwargs):
            start = time.time()
            try:
                wynik = funkcja(*args, **kwargs)
                return wynik
            except Exception as e:
                wpis = {
                    "timestamp": time.time(),
                    "funkcja": funkcja.__name__,
                    "args": repr(args),
                    "kwargs": repr(kwargs),
                    "typ_bledu": type(e).__name__,
                    "komunikat": str(e),
                    "czas_do_awarii_s": time.time() - start,
                }
                with open(log_path, "a", encoding="utf-8") as f:
                    f.write(json.dumps(wpis) + "\n")
                raise
        return wrapper
    return dekorator
```

Dekorator owija dowolną funkcję SUT, nie zmieniając jej logiki. Każdy wyjątek jest zapisywany w pliku JSON Lines: łatwym do wczytania i policzenia wskaźników.

Czego unikać

Logowania awarii w sposób, który sam się myli: wyłapywanie `except Exception` i cichy `return None` bez logowania to ukrywanie awarii, nie mierzenie niezawodności. Jeśli program połyka wyjątki bez śladu, wasze dane będą zaniżone, a wskaźniki fałszywie dobre.

Wskazówka

Z zebranych wpisów policzycie dziś wstępne $MTTF$ (średni czas działania między kolejnymi awariami w logu) oraz wstępną gęstość defektów (liczba unikalnych przyczyn awarii / tysiąc linii kodu SUT). To będzie wasz punkt odniesienia (*baseline*) do porównań w kolejnych tygodniach, gdy liczba wykrytych usterek będzie rosła wraz z testami strukturalnymi i funkcjonalnymi.

Przykład obliczeniowy: od logu do MTTF

Załóżmy, że podczas 100 wywołań funkcji SUT zarejestrowaliście w logu 4 awarie, w następujących momentach (czas w sekundach od startu testów):

$$t_1 = 12\text{s}, \quad t_2 = 47\text{s}, \quad t_3 = 53\text{s}, \quad t_4 = 118\text{s}$$

Krok 1: czasy między kolejnymi awariami:

$$\Delta_1 = 12, \quad \Delta_2 = 47 - 12 = 35, \quad \Delta_3 = 53 - 47 = 6, \quad \Delta_4 = 118 - 53 = 65$$

Krok 2: MTTF jako średnia tych odstępów:

$$MTTF = \frac{12 + 35 + 6 + 65}{4} = \frac{118}{4} = 29{,}5\text{s}$$

Krok 3: gęstość defektów, jeśli SUT ma np. 400 linii kodu (0,4 KLOC):

$$\text{gęstość} = \frac{4 \text{ usterki}}{0{,}4 \text{ KLOC}} = 10 \text{ usterek / KLOC}$$

Zwróćcie uwagę, że awarie nie są rozłożone równomiernie: odstęp $\Delta_3 = 6$s sugeruje, że w tym miejscu mogliście trafić na skupisko powiązanych usterek albo jedną usterkę aktywowaną wielokrotnie pod rząd. To właśnie tego typu obserwacje zapisujecie we wnioskach do raportu.

4 Zadanie projektowe na dziś

1. **Utwórzcie zespół** (2–3 osoby) i założcie wspólne repozytorium (Git).
2. **Odbierzcie SUT** od prowadzącego: system testowany, z którym będziecie pracować przez cały semestr (opis w Załączniku A). Każdy zespół dostaje inny wariant tego samego systemu.
3. **Uruchomcie środowisko** i upewnijcie się, że SUT działa (podstawowe wywołania przechodzą bez błędu).
4. **Zaimplementujcie mechanizm logowania awarii** (dekorator/wrapper jak w przykładzie, albo odpowiednik we własnym języku).
5. **Przeprowadźcie 20–30 eksploracyjnych wywołań** funkcji SUT z różnymi danymi wejściowymi (w tym nietypowymi: puste, ujemne, bardzo duże, znaki specjalne).
6. **Policzcie wstępne wskaźniki**: liczba awarii, wstępne MTTF, gęstość defektów.

Co trafia do raportu

Na koniec dzisiejszych zajęć każdy zespół przygotowuje krótką notatkę (0,5–1 strony) zawierającą:

- nazwę i krótki opis wybranego SUT,
- link do repozytorium,
- listę znalezionych dziś awarii (typ, dotkliwość, krótki opis),
- wstępne wartości: liczba wywołań, liczba awarii, MTTF, gęstość defektów,
- jedno zdanie wniosku: co z profilu operacyjnego SUT wydaje się najbardziej ryzykowne.

Ta notatka jest pierwszym elementem raportu końcowego: będzie rozbudowywana na kolejnych zajęciach.

5 Pytania kontrolne

1. Czym różni się fault od failure? Podaj przykład usterki, która nigdy nie ujawni się jako awaria.
2. Dlaczego dwa programy z taką samą liczbą usterek mogą mieć różną niezawodność?
3. Co oznacza dostępność na poziomie 99,9% w godzinach przestoju rocznie?
4. Dlaczego profil operacyjny jest istotny przy planowaniu testów?
5. Jakie dane musi zawierać wpis w logu awarii, żeby awaria była powtarzalna (reprodukowalna)?
6. Czym różni się awaria krytyczna od poważnej? Podaj przykład własny, inny niż w materiale.
7. Mając cztery awarie w czasach 12s, 47s, 53s, 118s, pokaż jak policzyć MTTF i podaj wynik.

6 Przygotowanie do zajęć nr 2 (blok P2)

Zajęcia nr 2 i 3 dotyczą testów strukturalnych: pokrycia instrukcji, gałęzi, warunków (MC/DC) oraz testów mutacyjnych. Na następne zajęcia:

- mieć działające środowisko SUT z zajęć 1,
- zainstalować narzędzie do pomiaru pokrycia kodu dla wybranego języka (dla Pythona: `coverage.py` oraz `pytest-cov`),
- przejrzeć pobieżnie kod SUT i zorientować się w jego strukturze (moduły, funkcje, rozgałęzienia).

Załącznik A: System testowany (SUT)

Na dzisiejszych zajęciach otrzymacie od prowadzącego wariant **systemu zarządzania kontem bankowym**: zakładanie konta, wpłata, wypłata, przelew, naliczanie odsetek, dzienny limit wypłat, blokada i odblokowanie konta, historia transakcji, wyciąg miesięczny. Każdy zespół pracuje na własnym wariancie tego samego systemu przez cały semestr.

Uwaga

System zawiera celowo ukryte usterki, o których nie wiecie: to jest właśnie przedmiot waszego projektu. Sens całego semestru polega na ich samodzielnym wykrywaniu różnymi technikami

testowania: dziś testami eksploracyjnymi, a w kolejnych tygodniach testami strukturalnymi, funkcjonalnymi, niefunkcjonalnymi i analizą statyczną. Nie szukajcie ”podpowiedzi” w kodzie ani w dokumentacji projektu: potraktujcie SUT tak, jakbyście dostali go od klienta bez żadnej wiedzy o jego wewnętrznych błędach.